

Narzędzia i Aplikacje Jakarta EE

Jakarta Servlet

Michał Wójcik

Servlet

Servlety to podstawowe komponenty w aplikacji webowej:

- jest to zwykła klasa języka Java implementująca interfejs `Servlet`,
- najczęściej mamy do czynienia z `HttpServlet` dla żądań HTTP,
- konfiguracja w deskryptorze `web.xml`,
- może mieć adnotację `@WebServlet`, wtedy nie trzeba go rejestrować w deskryptorze,
- pozwalają na generowanie odpowiedzi dowolnego typu: tekstowej (plik tekstowy, strona HTML, dokument JSON/XML), binarnej (np. obrazki, dokumenty PDF).

Servlet - przykład

```java
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String languageHeader = request.getHeader("Accept-Language");

        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();

        out.print("<h1>");
        out.print(String.format("Hello! %s", name));
        out.print("</h1>");
    }

}
```

Cykl życia

Cykl życia servletu:

1. jeśli nie istnieje instancja servletu, kontener webowy:
    1. ładuje klasę servletu,
    2. tworzy instancję tej klasy,
    3. wywołuje metodę `init()` servletu (inicjalizacja);
2. wywołuje metody obsługi żądań servletu `doPost()`, `doGet()`, `doPut()`, itd,
3. przy niszczeniu servletu wywoływana jest metoda `destroy()`.

Uwaga: dla jednego servletu *zazwyczaj* występuje tylko jedna jego instancja w kontenerze, z której korzysta wiele wątków jednocześnie.

Deklaracja w deskryptorze

Zamiast korzystania z adnotacji można skorzystać z konfiguracji w deskryptorze `web.xml`:

```
<servlet>
    <servlet-name>helloServlet</servlet-name>
    <servlet-class>pl.edu.pg.eti.kask.hello.HelloServlet</servlet-class>
</servlet>
```

```
<servlet-mapping>
    <servlet-name>helloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

Konfiguracja początkowa servletu

```
@WebServlet(urlPatterns = {"/hello"}, initParams = {
                @WebInitParam(name = "welcomeText", value = "Hello! %s")
})
public class HelloServlet extends HttpServlet {

    private String welcomeText;

    @Override
    public void init(ServletConfig config) throws ServletException {
        welcomeText = config.getInitParameter("welcomeText");
        super.init(config);
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String languageHeader = request.getHeader("Accept-Language");

        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();

        out.print("<h1>");
        out.print(String.format(welcomeText, name));
        out.print("</h1>");
    }

}
```

Konfiguracja w deskryptorze

Pierwszeństwo przed adnotacjami zawsze ma konfiguracja w deskryptorze.

```
<servlet>
    <servlet-name>helloServlet</servlet-name>
    <servlet-class>pl.edu.pg.eti.kask.hello.HelloServlet</servlet-class>
    <init-param>
        <param-name>welcomeText</param-name>
        <param-value>Hello! %s</param-value>
    </init-param>
</servlet>
```

```
<servlet-mapping>
    <servlet-name>helloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

Wgrywanie plików

Wgrywanie plików wymaga wykorzystania kodowania danych `multipart/form-data` zamiast domyślnego `application/x-www-form-urlencoded`.

```
@WebServlet("/upload")
@MultipartConfig
public class UploadServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        Part file = request.getPart("image");
    }

}
```

Współdzielenie obiektów

Kontener udostępnia mechanizm **współdzielenia** obiektów na różnych **poziomach**:

- aplikacji webowej,
- sesji,
- żądania.

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException,IOException{
        ///...
}
```

```
request.getServletContext().setAttribute("name", new Object());
Object object = request.getServletContext().getAttribute("name");
```

```
request.getSession().setAttribute("name", new Object());
Object object = request.getSession().getAttribute("name");
```

```
request.setAttribute("name", new Object());
Object object = request.getAttribute("name");
```

Zdarzenia w kontenerze

Można **nasłuchiwać** zdarzeń związanych z cyklem życia servletów i współdzielonymi obiektami poprzez adnotację `@WebListener` oraz implementację odpowiedniego interfejsu (`EventListener`):

- kontekst aplikacji webowej:
  - tworzenie, niszczenie - `ServletContextListener`:
  - tworzenie, niszczenie, modyfikacje atrybutów - `ServletContextAttributeListener`;
- sesja:
  - tworzenie, kończenie - `HttpSessionListener`,
  - aktywacja, pasywacja - `HttpSessionActivationListener`,
  - modyfikacje atrybutów `HttpSessionAttributeListener`;
- żądanie:
  - rozpoczęcie i zakończenie przetwarzania - `ServletRequestListener`,
  - modyfikacje atrybutów - `ServletRequestAttributeListener`.

Zdarzenia w kontenerze - przykład

```
@WebListener
public class SessionListener implements HttpSessionListener {

    @Override
    public void sessionCreated(HttpSessionEvent event) {
        event.getSession().setAttribute("name", new Object());
    }

}
```

Korzystając z adnotacji, nie można określić kolejności wykonywania obsługi zdarzeń.

Definicja obsługi zdarzeń w deskryptorze

Wykorzystanie deskryptora `web.xml` pozwala na zdefiniowanie kolejności, w jakiej będą wykonywane zdarzenia.

```
<listener>
    <listener-class>pl.edu.pg.eti.kask.hello.CreateResource</listener-class>
</listener>
```

```
<listener>
    <listener-class>pl.edu.pg.eti.kask.hello.FillData</listener-class>
</listener>
```

Filtry

Filtry pozwalają na podejmowanie akcji w komunikacji klient serwer podczas przetwarzania żądania:

- implementują interfejs `Filter`,
- mogą dziedziczyć po klasie `HttpFilter` dla żądań HTTP,
- wykonywane są sekwencyjnie w postaci łańcucha wywołań.

```
@WebFilter("/*")
public class SomeFilter extends HttpFilter {

    @Override
    public void doFilter(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {
        //do before
        chain.doFilter(request, response);
        //do after
    }

}
```

Filtry - wykorzystanie

Przykładowe / sugerowane przeznaczenie filtrów:

- uwierzytelnianie,
- autoryzacja,
- szyfrowanie,
- kompresja danych,
- weryfikacja i modyfikacja danych.

Filtry - przykład

Parametry żądania nie mogą być modyfikowane podczas jego obsługi. Możliwe jest nadal modyfikowanie atrybutów żądania.

```
@WebFilter("/form")
public class ConvertFilter extends HttpFilter {

    @Override
    public void doFilter(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {
        String number = request.getParameter("number");
        request.setAttribute("number", Integer.parseInt(number));
        chain.doFilter(request, response);
    }

}
```

Definicja filtrów w deskryptorze

Korzystając z adnotacji, nie jest możliwe ustawienie kolejności uruchamiania filtrów. Konfiguracja w deskryptorze `web.xml` pozwala na ustalenie kolejności:

```
<filter>
    <filter-name>authenticationFilter</filter-name>
    <filter-class>pl.edu.pg.eti.kask.hello.AuthenticationFilter</filter-class>
</filter>
```

```
<filter-mapping>
    <filter-name>authenticationFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

Servlety dzisiaj

Deweloper aplikacji rzadko pracuje bezpośrednio z servletami, filtrami czy obsługą zdarzeń:

- typowo używa frameworka, który ułatwia obsługę żądań,
- przydają się, kiedy trzeba zrobić coś nietypowego (coś, czego nie przewidzieli autorzy frameworka),
- integrują się z innymi rozwiązaniami, np. CDI, EJB,
- są podstawowym mechanizmem obsługi żądań, na którym frameworki webowe nadbudowują swoje funkcjonalności.

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/servlet/6.1/jakarta-servlet-spec-6.1

Uzupełnienie informacji:

- https://jakarta.ee/learn/docs/jakartaee-tutorial/current/web/webapp/webapp.html
- https://www.baeldung.com/intro-to-servlets
- https://www.baeldung.com/java-servlets-containers-intro
- https://www.baeldung.com/upload-file-servlet